

L4: LINEAR CLASSIFIERS



Classification: Problem Statement

- In regression, we are modeling the relationship between a continuous input variable $\mathbf{x}$ and a continuous target variable t .
- In classification, the input variable $\mathbf{x}$ may still be continuous, but the target variable is discrete.
- In the simplest case, t can have only 2 values.

e.g., Let $t = +1 \leftrightarrow \mathbf{x}$ assigned to C_1

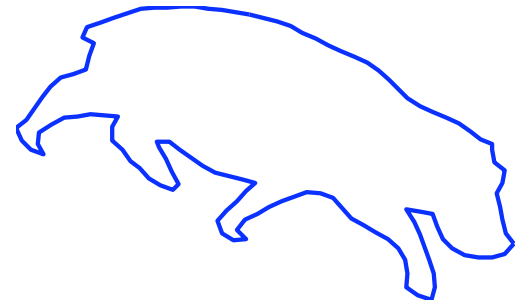
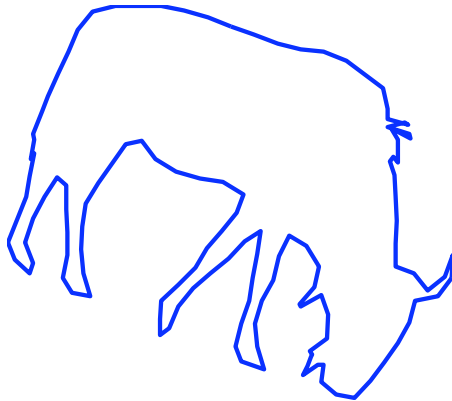
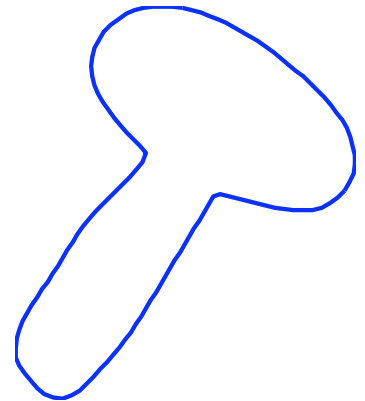
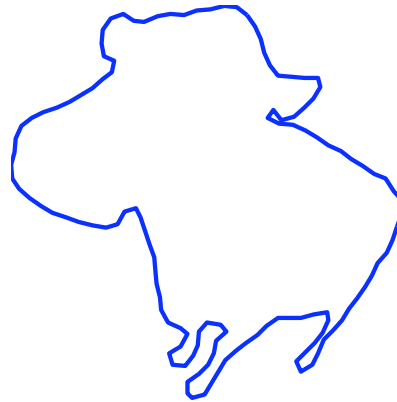
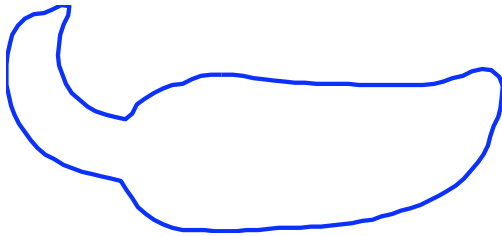
$t = -1 \leftrightarrow \mathbf{x}$ assigned to C_2

Example Problem

4

Probability & Bayesian Inference

□ Animal or Vegetable?



Discriminative Classifiers

- If the conditional distributions are normal, the best thing to do is to estimate the parameters of these distributions and use Bayesian decision theory to classify input vectors. Decision boundaries are generally quadratic.
- However if the conditional distributions are not exactly normal, this **generative** approach will yield sub-optimal results.
- Another alternative is to build a **discriminative** classifier, that focuses on modeling the decision boundary directly, rather than the conditional distributions themselves.

Linear Models for Classification

6

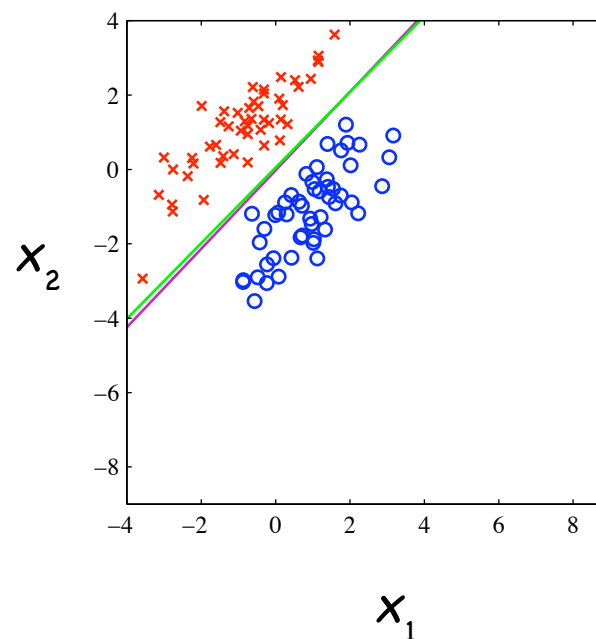
Probability & Bayesian Inference

- Linear models for classification separate input vectors into classes using linear (hyperplane) *decision boundaries*.

- Example:

2D Input vector $\mathbf{x}$

Two discrete classes C_1 and C_2



Two Class Discriminant Function

7

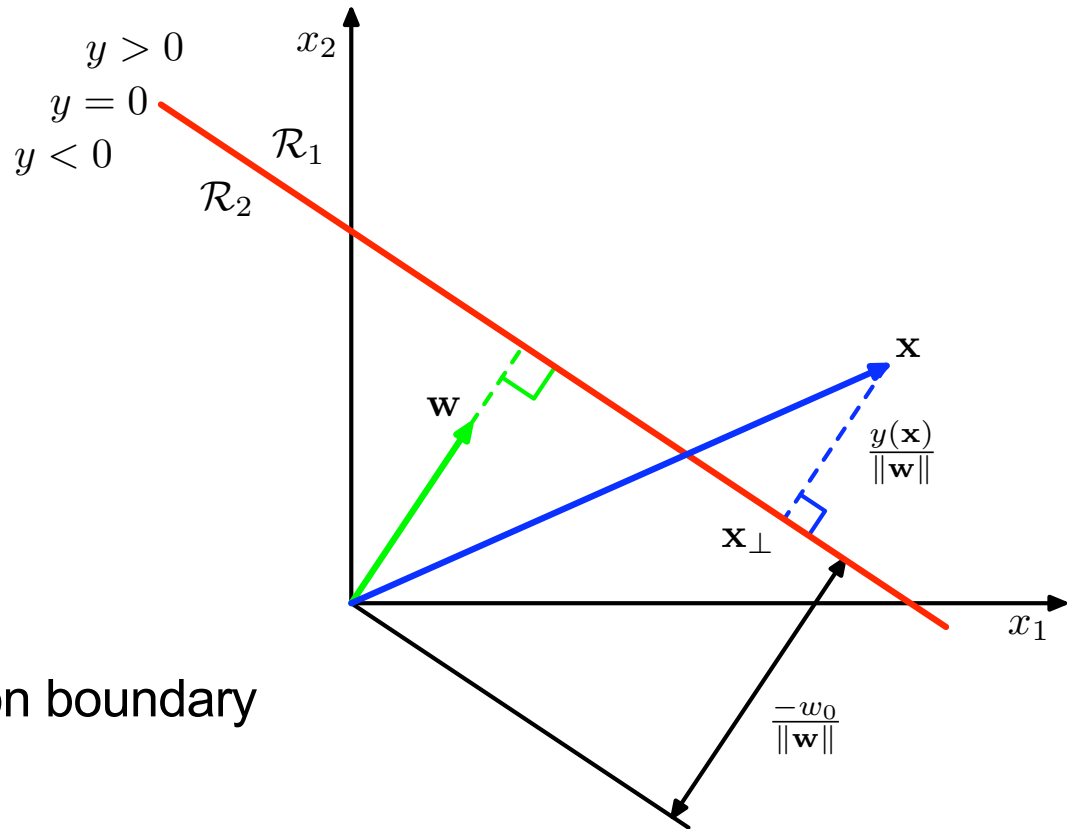
Probability & Bayesian Inference

$$y(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + w_0$$

$y(\mathbf{x}) \geq 0 \rightarrow \mathbf{x}$ assigned to C_1

$y(\mathbf{x}) < 0 \rightarrow \mathbf{x}$ assigned to C_2

Thus $y(\mathbf{x}) = 0$ defines the decision boundary



Two-Class Discriminant Function

8

Probability & Bayesian Inference

$$y(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + w_0$$

$y(\mathbf{x}) \geq 0 \rightarrow \mathbf{x}$ assigned to C_1

$y(\mathbf{x}) < 0 \rightarrow \mathbf{x}$ assigned to C_2

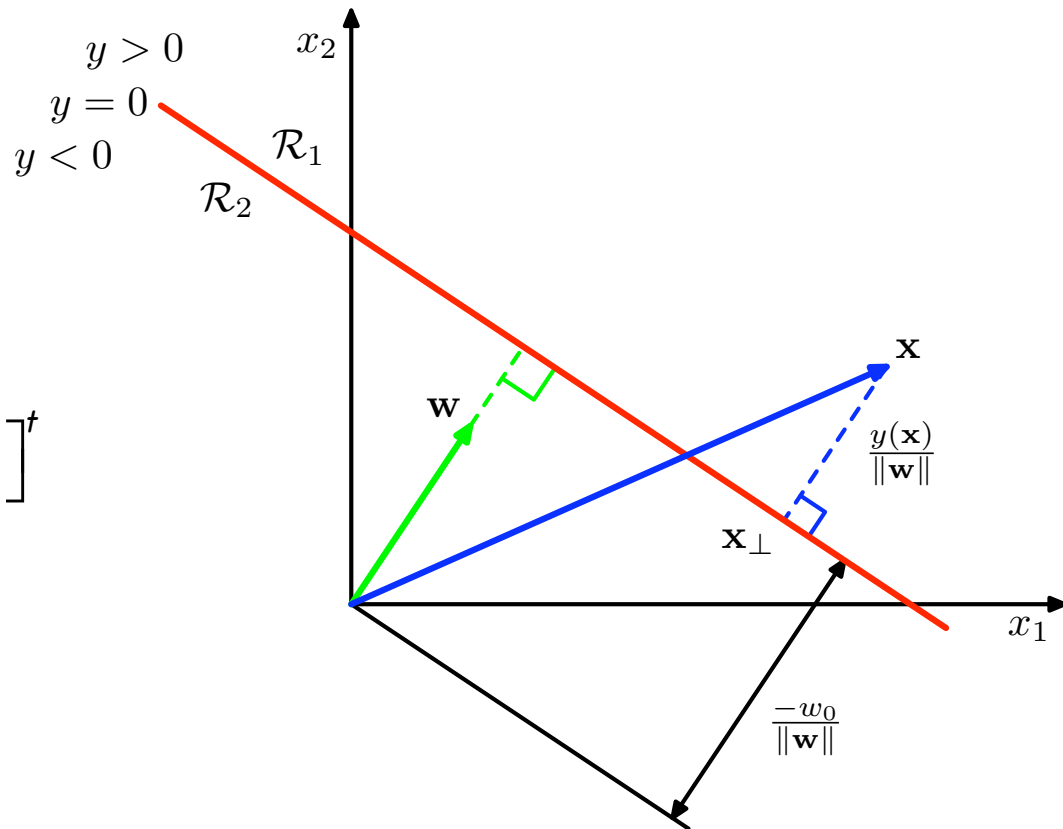
For convenience, let

$$\mathbf{w} = [w_1 \dots w_M]^t \Rightarrow [w_0 \ w_1 \dots w_M]^t$$

and

$$\mathbf{x} = [x_1 \dots x_M]^t \Rightarrow [1 \ x_1 \dots x_M]^t$$

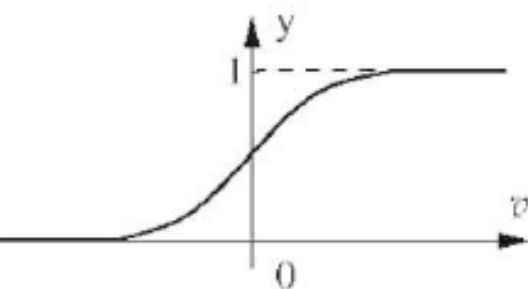
So we can express $y(\mathbf{x}) = \mathbf{w}^t \mathbf{x}$



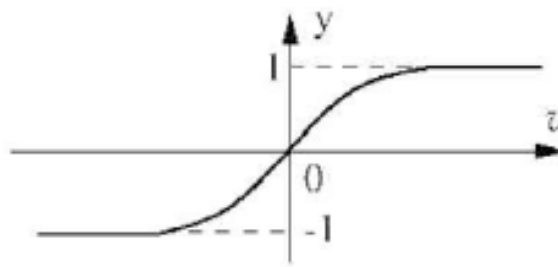
Generalized Linear Models

- For classification problems, we want y to be a predictor of t . In other words, we wish to map the input vector into one of a number of discrete classes, or to posterior probabilities that lie between 0 and 1.
- For this purpose, it is useful to elaborate the linear model by introducing a nonlinear activation function f , which typically will constrain y to lie between -1 and 1 or between 0 and 1.

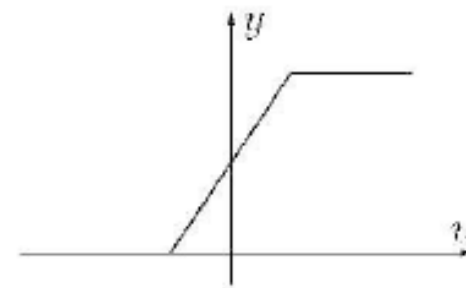
$$y(\mathbf{x}) = f(\mathbf{w}^t \mathbf{x} + w_0)$$



Log-sigmoid function



Tan-sigmoid function

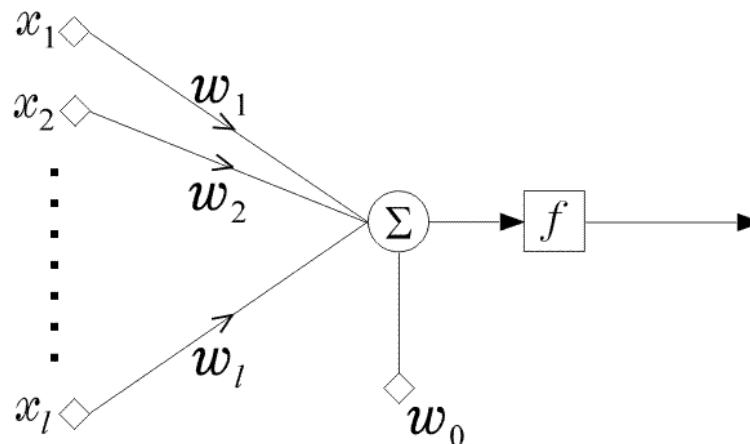


Linear function

The Perceptron

$$y(\mathbf{x}) = f(\mathbf{w}^t \mathbf{x} + w_0) \quad \begin{array}{l} y(\mathbf{x}) \geq 0 \rightarrow \mathbf{x} \text{ assigned to } C_1 \\ y(\mathbf{x}) < 0 \rightarrow \mathbf{x} \text{ assigned to } C_2 \end{array}$$

- A classifier based upon this simple generalized linear model is called a (single layer) **perceptron**.
- It can also be identified with an abstracted model of a neuron called the McCulloch Pitts model.



Parameter Learning

12

Probability & Bayesian Inference

- How do we learn the parameters of a perceptron?

Outline

13

Probability & Bayesian Inference

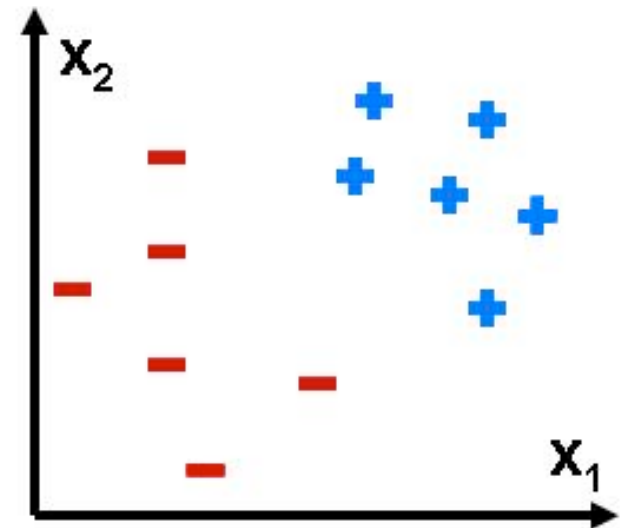
- **The Perceptron Algorithm**
- Least-Squares Classifiers
- Fisher's Linear Discriminant
- Logistic Classifiers

Case 1. Linearly Separable Inputs

14

Probability & Bayesian Inference

- For starters, let's assume that the training data is in fact perfectly linearly separable.
- In other words, there exists at least one hyperplane (one set of weights) that yields 0 classification error.
- We seek an algorithm that can automatically find such a hyperplane.



The Perceptron Algorithm

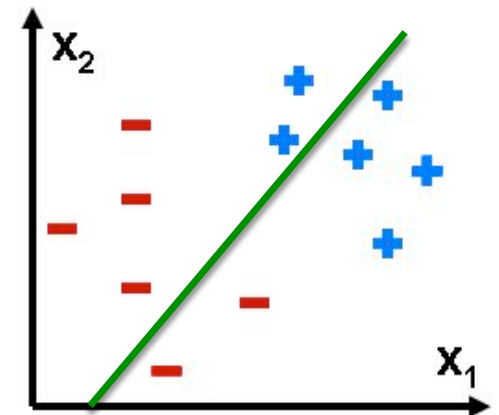
15

Probability & Bayesian Inference

- The perceptron algorithm was invented by Frank Rosenblatt (1962).
- The algorithm is iterative.
- The strategy is to start with a random **guess** at the weights $\mathbf{w}$, and to then iteratively change the weights to move the hyperplane in a direction that lowers the classification error.



Frank Rosenblatt (1928 – 1971)

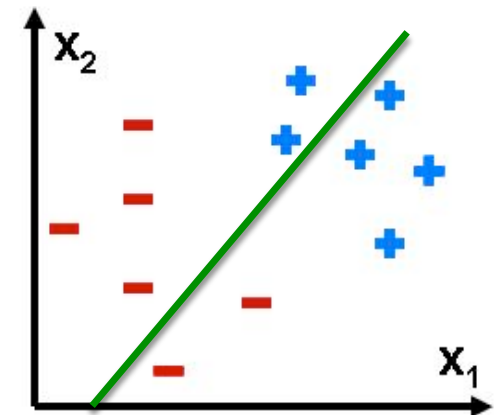


The Perceptron Algorithm

16

Probability & Bayesian Inference

- Note that as we change the weights continuously, the classification error changes in discontinuous, piecewise constant fashion.
- Thus we cannot use the classification error per se as our objective function to minimize.
- What would be a better objective function?



The Perceptron Criterion

17

Probability & Bayesian Inference

- Note that we seek $\mathbf{w}$ such that

$$\mathbf{w}^t \mathbf{x} \geq 0 \text{ when } t = +1$$

$$\mathbf{w}^t \mathbf{x} < 0 \text{ when } t = -1$$

- In other words, we would like

$$\mathbf{w}^t \mathbf{x}_n t_n \geq 0 \quad \forall n$$

- Thus we seek to minimize

$$E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^t \mathbf{x}_n t_n$$

where $\mathcal{M}$ is the set of misclassified inputs.

The Perceptron Criterion

18

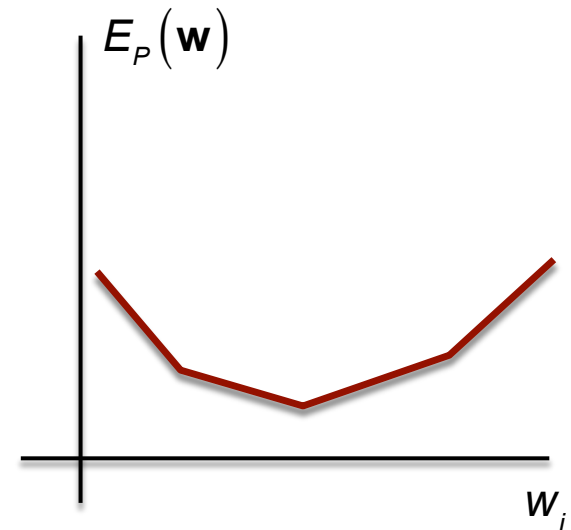
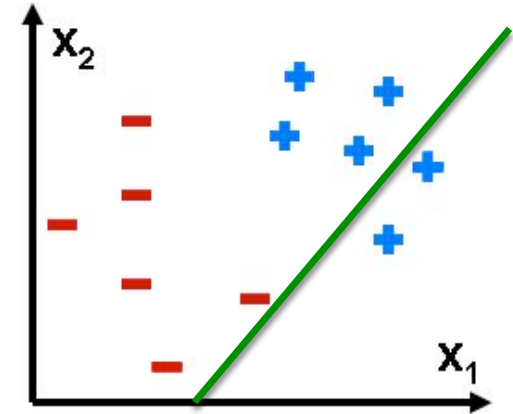
Probability & Bayesian Inference

$$E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^t \mathbf{x}_n t_n$$

where $\mathcal{M}$ is the set of misclassified inputs.

□ Observations:

- $E_P(\mathbf{w})$ is always non-negative.
- $E_P(\mathbf{w})$ is continuous and piecewise linear, and thus easier to minimize.



The Perceptron Algorithm

19

Probability & Bayesian Inference

$$E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^t \mathbf{x}_n t_n$$

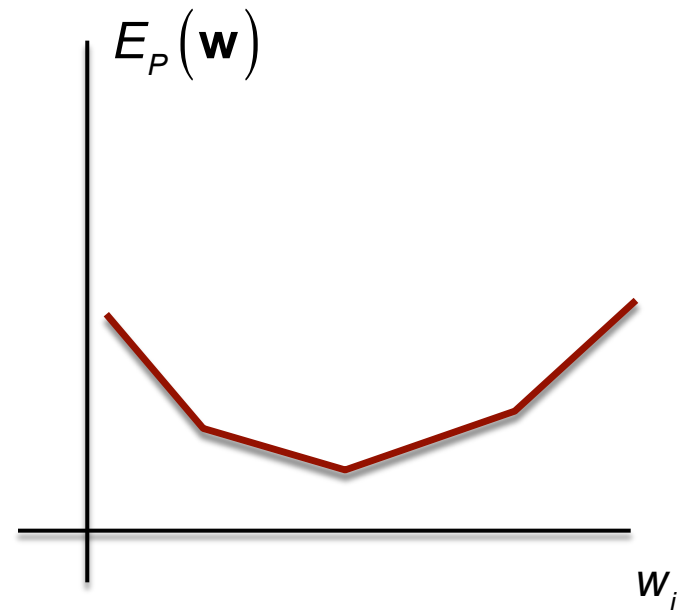
where $\mathcal{M}$ is the set of misclassified inputs.

$$\frac{dE_P(\mathbf{w})}{d\mathbf{w}} = - \sum_{n \in \mathcal{M}} \mathbf{x}_n t_n$$

where the derivative exists.

□ Gradient descent:

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla E_P(\mathbf{w}) = \mathbf{w}^{\tau} + \eta \sum_{n \in \mathcal{M}} \mathbf{x}_n t_n$$



The Perceptron Algorithm

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla E_P(\mathbf{w}) = \mathbf{w}^t + \eta \sum_{n \in \mathcal{M}} \mathbf{x}_n t_n$$

- Why does this make sense?
 - ▣ If an input from $C_1 (t = +1)$ is misclassified, we need to make its projection on $\mathbf{w}$ more positive.
 - ▣ If an input from $C_2 (t = -1)$ is misclassified, we need to make its projection on $\mathbf{w}$ more negative.

The Perceptron Algorithm

21

Probability & Bayesian Inference

□ The algorithm can be implemented sequentially:

□ Repeat until convergence:

■ For each input $(\mathbf{x}_n, t_n)$:

■ If it is correctly classified, do nothing

■ If it is misclassified, update the weight vector to be

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} + \eta \mathbf{x}_n t_n$$

■ Note that this will lower the contribution of input n to the objective function:

$$-\left(\mathbf{w}^{(\tau)}\right)^t \mathbf{x}_n t_n \rightarrow -\left(\mathbf{w}^{(\tau+1)}\right)^t \mathbf{x}_n t_n = -\left(\mathbf{w}^{(\tau)}\right)^t \mathbf{x}_n t_n - \eta \left(\mathbf{x}_n t_n\right)^t \mathbf{x}_n t_n < -\left(\mathbf{w}^{(\tau)}\right)^t \mathbf{x}_n t_n.$$

Not Monotonic

- While updating with respect to a misclassified input n will lower the error for that input, the error for other misclassified inputs may increase.
- Also, new inputs that had been classified correctly may now be misclassified.
- The result is that the perceptron algorithm is not guaranteed to reduce the total error monotonically at each stage.

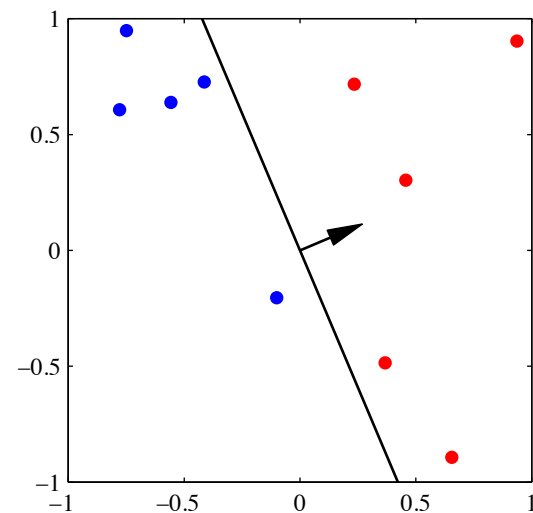
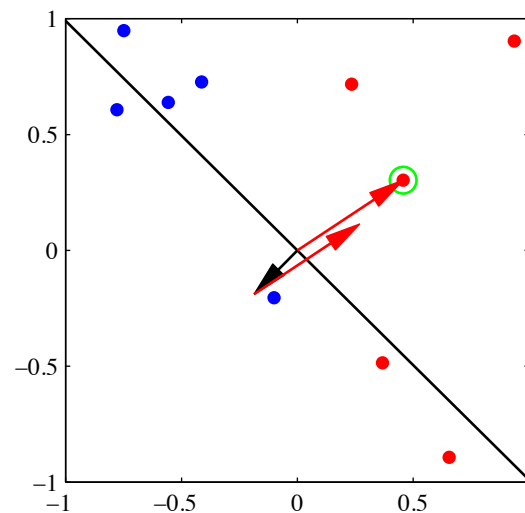
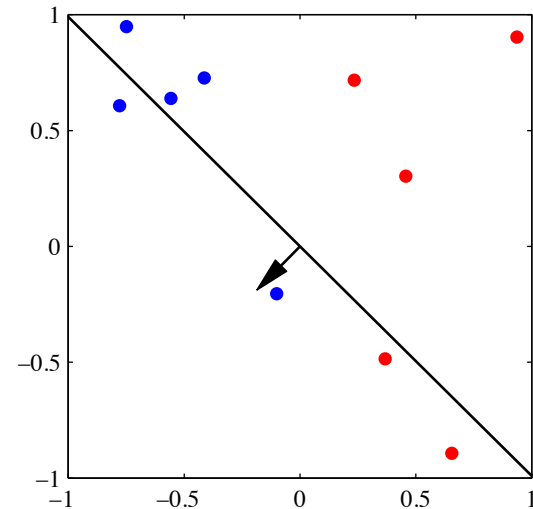
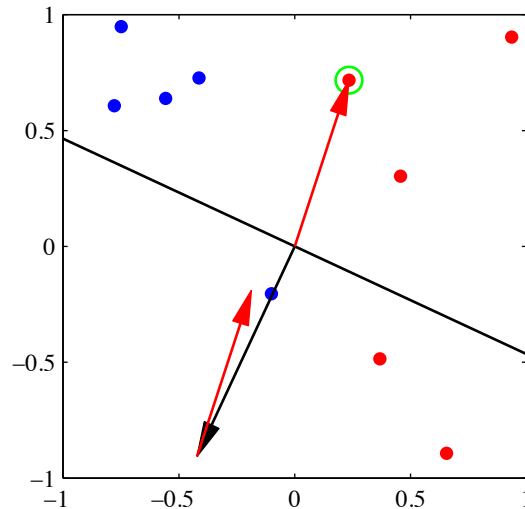
The Perceptron Convergence Theorem

- Despite this non-monotonicity, **if in fact the data are linearly separable, then the algorithm is guaranteed to find an exact solution in a finite number of steps** (Rosenblatt, 1962).

Example

24

Probability & Bayesian Inference



Numerical example

Compute the perceptron and MSE solution for the dataset

- $X1 = [(1,6), (7,2), (8,9), (9,9)]$
- $X2 = [(2,1), (2,2), (2,4), (7,1)]$

Perceptron learning

- Assume $\eta = 0.1$ and an online update rule
- Assume $a(0) = [0.1, 0.1, 0.1]$
- SOLUTION

- Normalize the dataset
- Iterate through all the examples and update $a(k)$ on the ones that are misclassified

- $Y(1) \Rightarrow [1 \ 1 \ 6] * [0.1 \ 0.1 \ 0.1]^T > 0 \Rightarrow$ no update
- $Y(2) \Rightarrow [1 \ 7 \ 2] * [0.1 \ 0.1 \ 0.1]^T > 0 \Rightarrow$ no update

...

- $Y(5) \Rightarrow [-1 \ -2 \ -1] * [0.1 \ 0.1 \ 0.1]^T < 0 \Rightarrow$ update $a(1) = [0.1 \ 0.1 \ 0.1] + \eta[-1 \ -2 \ -1] = [0 \ -0.1 \ 0]$
- $Y(6) \Rightarrow [-1 \ -2 \ -2] * [0 \ -0.1 \ 0]^T > 0 \Rightarrow$ no update

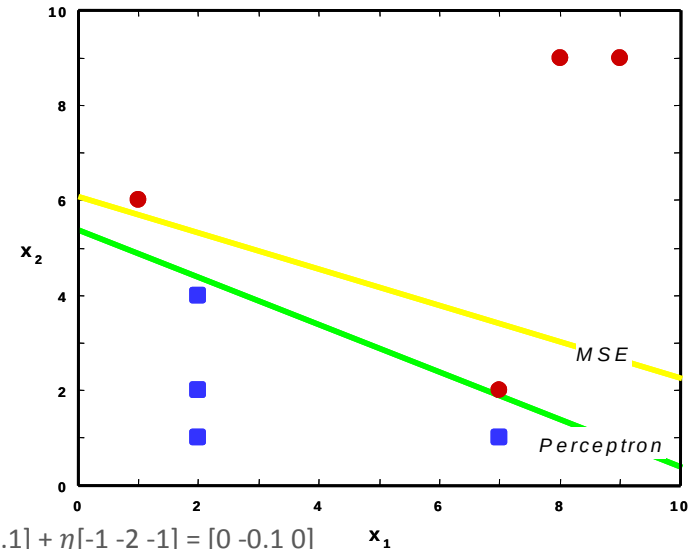
....

- $Y(1) \Rightarrow [1 \ 1 \ 6] * [0 \ -0.1 \ 0]^T < 0 \Rightarrow$ update $a(2) = [0 \ -0.1 \ 0] + \eta[1 \ 1 \ 6] = [0.1 \ 0 \ 0.6]$
- $Y(2) \Rightarrow [1 \ 7 \ 2] * [0.1 \ 0 \ 0.6]^T > 0 \Rightarrow$ no update

...

- In this example, the perceptron rule converges after 175 iterations to $a = [-3.5 \ 0.3 \ 0.7]$
- To convince yourself this is a solution, compute $Y a$ (you will find out that all terms are non-negative)

$$Y = \begin{bmatrix} 1 & 1 & 6 \\ 1 & 7 & 2 \\ 1 & 8 & 9 \\ 1 & 9 & 9 \\ -1 & -2 & -1 \\ -1 & -2 & -2 \\ -1 & -2 & -4 \\ -1 & -7 & -1 \end{bmatrix}$$



Practical Limitations

- The Perceptron Convergence Theorem is an important result. However, there are practical limitations:
 - ▣ Convergence may be slow
 - ▣ If the data are not separable, the algorithm will not converge.
 - ▣ We will only know that the data are separable once the algorithm converges.
 - ▣ The solution is in general not unique, and will depend upon initialization, scheduling of input vectors, and the learning rate η .

Generalization to inputs that are not linearly separable.

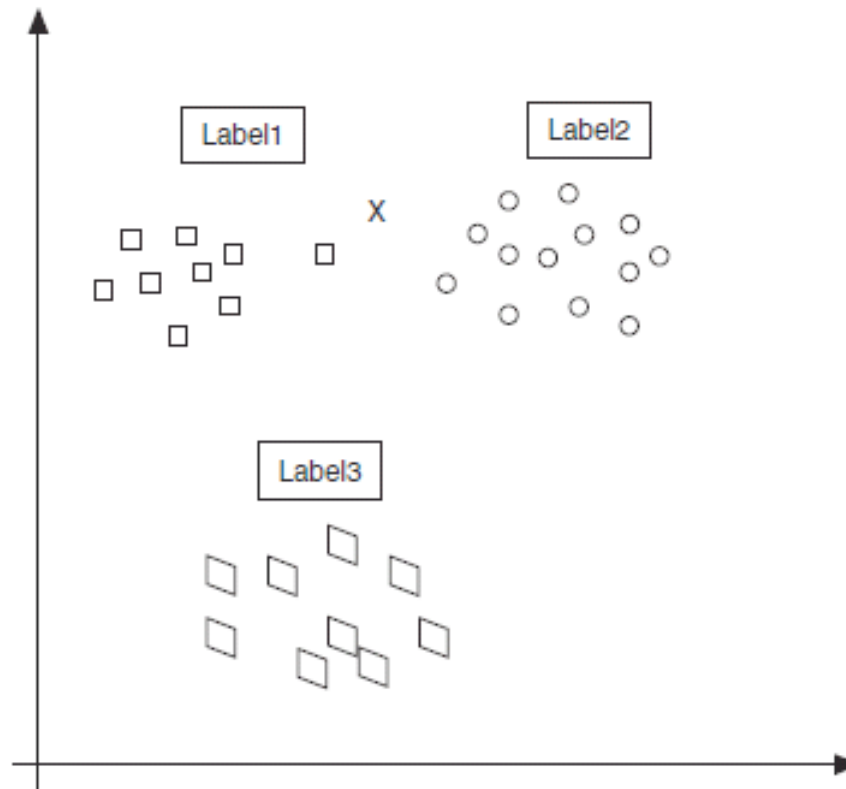
- The single-layer perceptron can be generalized to yield good linear solutions to problems that are not linearly separable.
- Example: The Pocket Algorithm (Gal 1990)
 - ▣ Idea:
 - Run the perceptron algorithm
 - Keep track of the weight vector $\mathbf{w}^*$ that has produced the
 - . best classification error achieved so far
 - . |

Generalization to Multiclass Problems

28

Probability & Bayesian Inference

- How can we use perceptrons, or linear classifiers in general, to classify inputs when there are $K > 2$ classes?

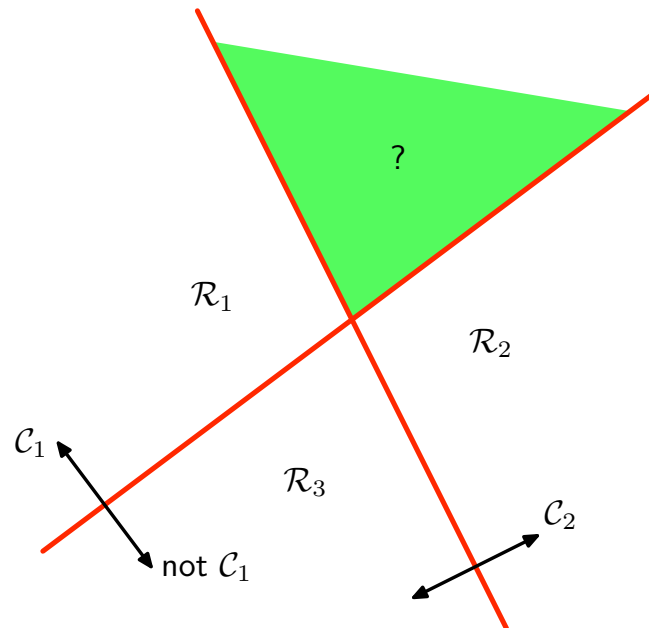


$K > 2$ Classes

29

Probability & Bayesian Inference

- Idea #1: Just use $K-1$ discriminant functions, each of which separates one class C_k from the rest. (One-versus-the-rest classifier.)
- Problem: Ambiguous regions

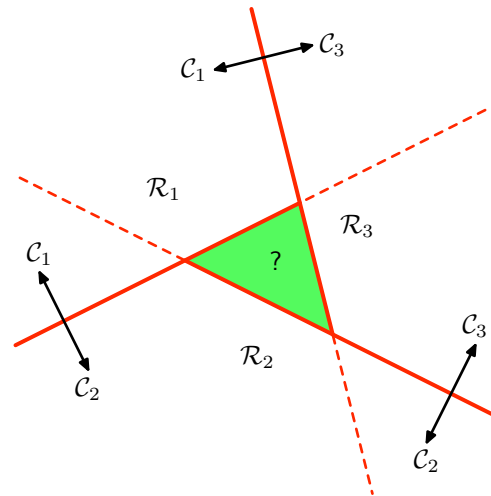


$K > 2$ Classes

30

Probability & Bayesian Inference

- Idea #2: Use $K(K-1)/2$ discriminant functions, each of which separates two classes C_j, C_k from each other. (One-versus-one classifier.)
- Each point classified by majority vote.
- Problem: Ambiguous regions



$K > 2$ Classes

31

Probability & Bayesian Inference

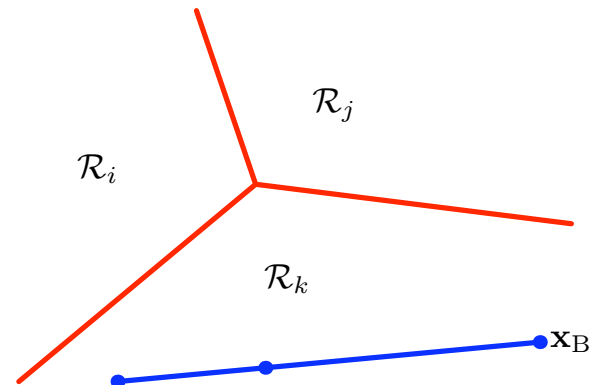
- Idea #3: Use K discriminant functions $y_k(\mathbf{x})$
- Use the **magnitude** of $y_k(\mathbf{x})$, not just the sign.

$$y_k(\mathbf{x}) = \mathbf{w}_k^t \mathbf{x}$$

$\mathbf{x}$ assigned to C_k if $y_k(\mathbf{x}) > y_j(\mathbf{x}) \forall j \neq k$

Decision boundary $y_k(\mathbf{x}) = y_j(\mathbf{x}) \rightarrow (\mathbf{w}_k - \mathbf{w}_j)^t \mathbf{x} + (\mathbf{w}_{k0} - \mathbf{w}_{j0}) = 0$

Results in decision regions that are simply-connected and convex.



Example: Kesler's Construction

32

Probability & Bayesian Inference

- The perceptron algorithm can be generalized to K -class classification problems.
- Example:
 - ▣ Kesler's Construction:
 - Allows use of the perceptron algorithm to simultaneously learn K separate weight vectors $\mathbf{w}_i$.
 - Inputs are then classified in Class i if and only if
$$\mathbf{w}_i^t \mathbf{x} > \mathbf{w}_j^t \mathbf{x} \quad \forall j \neq i$$
 - The algorithm will converge to an optimal solution if a solution exists, i.e., if all training vectors can be correctly classified according to this rule.

1-of-K Coding Scheme

33

Probability & Bayesian Inference

- When there are $K > 2$ classes, target variables can be coded using the 1-of-K coding scheme:

Input from Class $C_i \Leftrightarrow t = [0 \ 0 \ \dots 1 \ \dots 0 \ 0]^t$



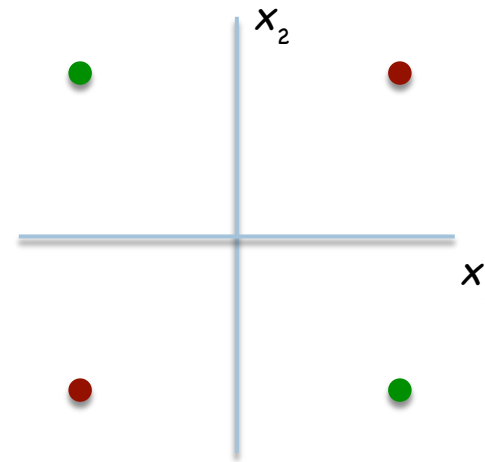
Element i

Computational Limitations of Perceptrons

34

Probability & Bayesian Inference

- Initially, the perceptron was thought to be a potentially powerful learning machine that could model human neural processing.
- However, Minsky & Papert (1969) showed that the single-layer perceptron could not learn a simple XOR function.
- This is just one example of a non-linearly separable pattern that cannot be learned by a single-layer perceptron.



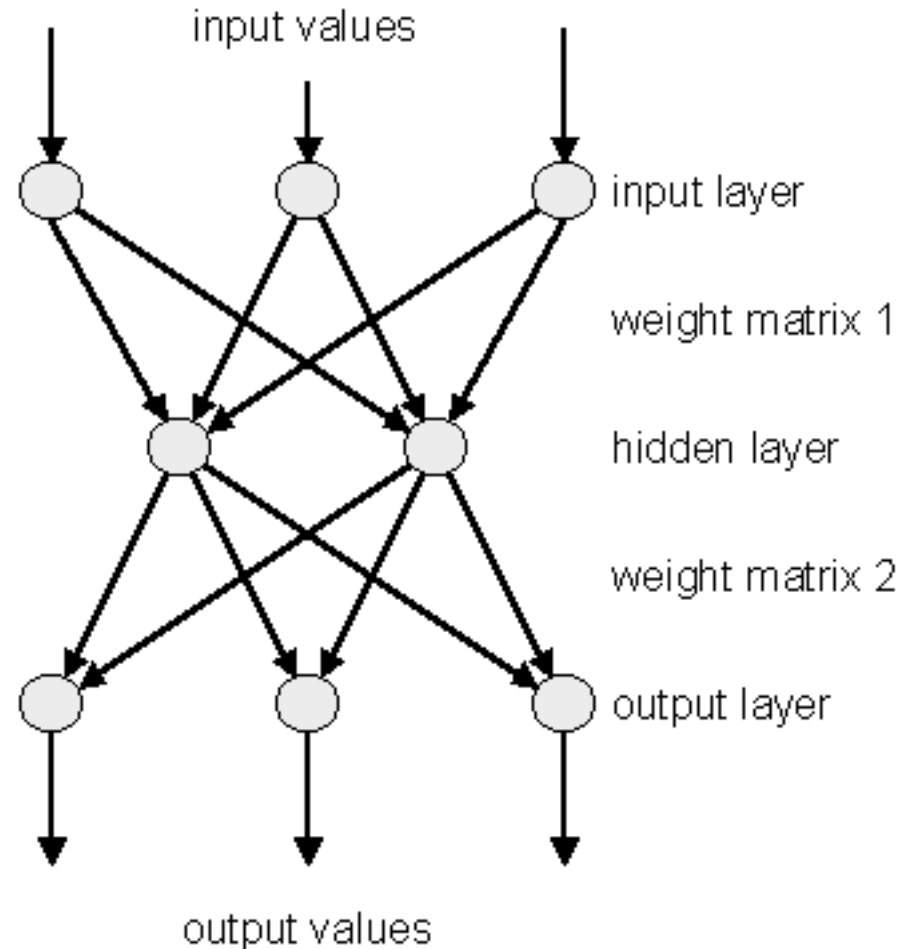
Marvin Minsky (1927 -)

Multi-Layer Perceptrons

35

Probability & Bayesian Inference

- Minsky & Papert's book was widely misinterpreted as showing that artificial neural networks were inherently limited.
- This contributed to a decline in the reputation of neural network research through the 70s and 80s.
- However, their findings apply only to single-layer perceptrons. Multi-layer perceptrons are capable of learning highly nonlinear functions, and are used in many practical applications.



Outline

36

Probability & Bayesian Inference

- The Perceptron Algorithm
- **Least-Squares Classifiers**
- Fisher's Linear Discriminant
- Logistic Classifiers

Dealing with Non-Linearly Separable Inputs

- The perceptron algorithm fails when the training data are not perfectly linearly separable.
- Let's now turn to methods for learning the parameter vector $\mathbf{w}$ of a perceptron (linear classifier) even when the training data are not linearly separable.

The Least Squares Method

38

Probability & Bayesian Inference

- In the least squares method, we simply fit the $(\mathbf{x}, t)$ observations with a hyperplane $y(\mathbf{x})$.
- Note that this is kind of a weird idea, since the t values are binary (when $K=2$), e.g., 0 or 1.
- However it can work pretty well.

Least Squares: Learning the Parameters

39

Probability & Bayesian Inference

Assume D – dimensional input vectors $\mathbf{x}$.

For each class $k \in 1 \dots K$:

$$y_k(\mathbf{x}) = \mathbf{w}_k^t \mathbf{x} + w_{k0}$$

$$\rightarrow \mathbf{y}(\mathbf{x}) = \tilde{\mathbf{W}}^t \tilde{\mathbf{x}}$$

where

$$\tilde{\mathbf{x}} = (1, \mathbf{x}^t)^t$$

$\tilde{\mathbf{W}}$ is a $(D+1) \times K$ matrix whose k th column is $\tilde{\mathbf{w}}_k = (w_0, \mathbf{w}_k^t)^t$

Learning the Parameters

40

Probability & Bayesian Inference

□ Method #2: Least Squares

$$\mathbf{y}(\mathbf{x}) = \tilde{\mathbf{W}}^t \tilde{\mathbf{x}}$$

Training dataset $(\mathbf{x}_n, \mathbf{t}_n)$, $n = 1, \dots, N$

where we use the 1-of- K coding scheme for $\mathbf{t}_n$

Let $\mathbf{T}$ be the $N \times K$ matrix whose n^{th} row is $\mathbf{t}_n^t$

Let $\tilde{\mathbf{X}}$ be the $N \times (D + 1)$ matrix whose n^{th} row is $\tilde{\mathbf{x}}_n^t$

Let $\mathbf{R}_D(\tilde{\mathbf{W}}) = \tilde{\mathbf{X}}\tilde{\mathbf{W}} - \mathbf{T}$

Then we define the error as $E_D(\tilde{\mathbf{W}}) = \frac{1}{2} \sum_{i,j} R_{ij}^2 = \frac{1}{2} \text{Tr} \left\{ \mathbf{R}_D(\tilde{\mathbf{W}})^t \mathbf{R}_D(\tilde{\mathbf{W}}) \right\}$

Setting derivative wrt $\tilde{\mathbf{W}}$ to 0 yields:

$$\tilde{\mathbf{W}} = (\tilde{\mathbf{X}}^t \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^t \mathbf{T} = \tilde{\mathbf{X}}^\dagger \mathbf{T}$$

Recall: $\frac{\partial}{\partial A} \text{Tr}(AB) = B^t.$

Numerical example

Compute the perceptron and MSE solution for the dataset

- $X1 = [(1,6), (7,2), (8,9), (9,9)]$
- $X2 = [(2,1), (2,2), (2,4), (7,1)]$

Perceptron learning

- Assume $\eta = 0.1$ and an online update rule
- Assume $a(0) = [0.1, 0.1, 0.1]$
- SOLUTION

- Normalize the dataset
- Iterate through all the examples and update $a(k)$ on the ones that are misclassified

- $Y(1) \Rightarrow [1 \ 1 \ 6] * [0.1 \ 0.1 \ 0.1]^T > 0 \Rightarrow$ no update
- $Y(2) \Rightarrow [1 \ 7 \ 2] * [0.1 \ 0.1 \ 0.1]^T > 0 \Rightarrow$ no update

...

- $Y(5) \Rightarrow [-1 \ -2 \ -1] * [0.1 \ 0.1 \ 0.1]^T < 0 \Rightarrow$ update $a(1) = [0.1 \ 0.1 \ 0.1] + \eta[-1 \ -2 \ -1] = [0 \ -0.1 \ 0]$
- $Y(6) \Rightarrow [-1 \ -2 \ -2] * [0 \ -0.1 \ 0]^T > 0 \Rightarrow$ no update

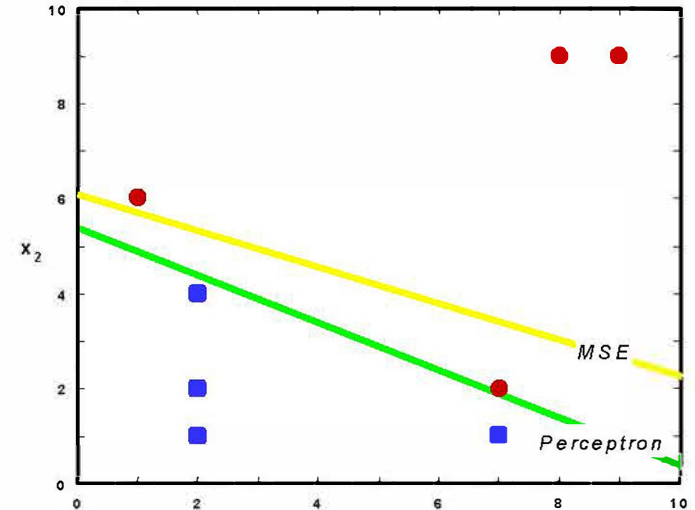
....

- $Y(1) \Rightarrow [1 \ 1 \ 6] * [0 \ -0.1 \ 0]^T < 0 \Rightarrow$ update $a(2) = [0 \ -0.1 \ 0] + \eta[1 \ 1 \ 6] = [0.1 \ 0 \ 0.6]$
- $Y(2) \Rightarrow [1 \ 7 \ 2] * [0.1 \ 0 \ 0.6]^T > 0 \Rightarrow$ no update

...

- In this example, the perceptron rule converges after 175 iterations to $a = [-3.5 \ 0.3 \ 0.7]$
- To convince yourself this is a solution, compute $Y a$ (you will find out that all terms are non-negative)

$$Y = \begin{bmatrix} 1 & 1 & 6 \\ 1 & 7 & 2 \\ 1 & 8 & 9 \\ 1 & 9 & 9 \\ -1 & -2 & -1 \\ -1 & -2 & -2 \\ -1 & -2 & -4 \\ -1 & -7 & -1 \end{bmatrix}$$

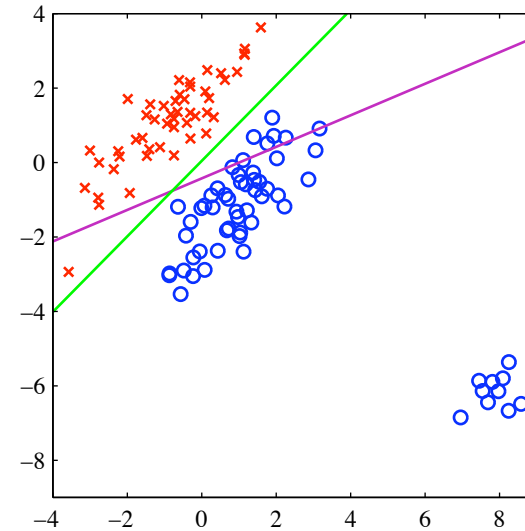
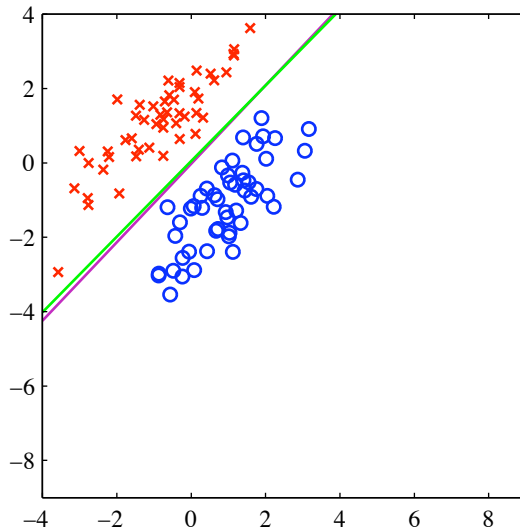


Problems with Least Squares

47

Probability & Bayesian Inference

□ Problem #1: Sensitivity to outliers

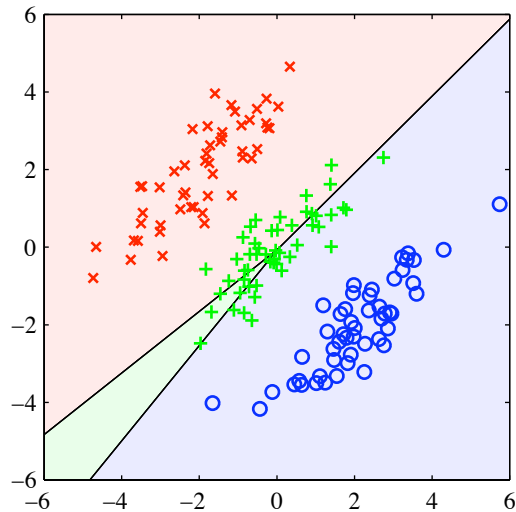


Problems with Least Squares

48

Probability & Bayesian Inference

- Problem #2: Linear activation function is not a good fit to binary data. This can lead to problems.



Outline

41

Probability & Bayesian Inference

- The Perceptron Algorithm
- Least-Squares Classifiers
- **Fisher's Linear Discriminant**
- Logistic Classifiers

Fisher's Linear Discriminant

42

Probability & Bayesian Inference

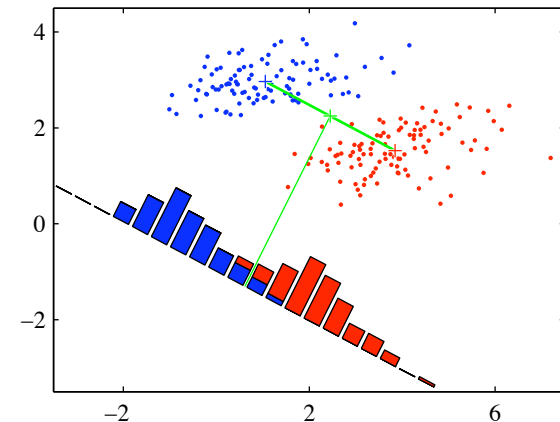
- Another way to view linear discriminants: find the 1D subspace that maximizes the separation between the two classes.

$$\text{Let } \mathbf{m}_1 = \frac{1}{N_1} \sum_{n \in C_1} \mathbf{x}_n, \quad \mathbf{m}_2 = \frac{1}{N_2} \sum_{n \in C_2} \mathbf{x}_n$$

For example, might choose $\mathbf{w}$ to maximize $\mathbf{w}^t (\mathbf{m}_2 - \mathbf{m}_1)$, subject to $\|\mathbf{w}\| = 1$

This leads to $\mathbf{w} \propto \mathbf{m}_2 - \mathbf{m}_1$

However, if conditional distributions are not isotropic, this is typically not optimal.



Fisher's Linear Discriminant

43

Probability & Bayesian Inference

Let $m_1 = \mathbf{w}^t \mathbf{m}_1$, $m_2 = \mathbf{w}^t \mathbf{m}_2$ be the conditional means on the 1D subspace.

Let $s_k^2 = \sum_{n \in C_k} (y_n - m_k)^2$ be the within-class variance on the subspace for class C_k

The Fisher criterion is then $J(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2}$

This can be rewritten as

$$J(\mathbf{w}) = \frac{\mathbf{w}^t \mathbf{S}_B \mathbf{w}}{\mathbf{w}^t \mathbf{S}_W \mathbf{w}}$$

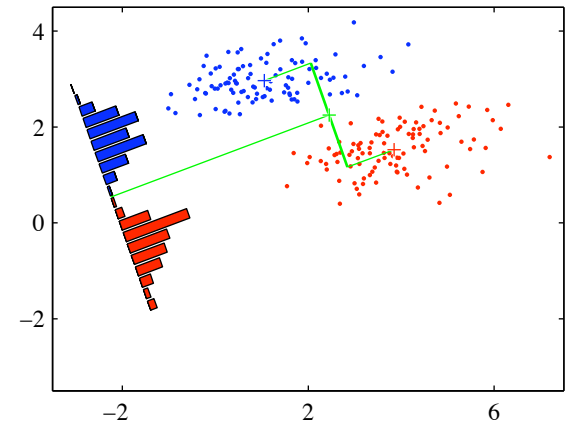
where

$\mathbf{S}_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^t$ is the between-class variance

and

$\mathbf{S}_W = \sum_{n \in C_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^t + \sum_{n \in C_2} (\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^t$ is the within-class variance

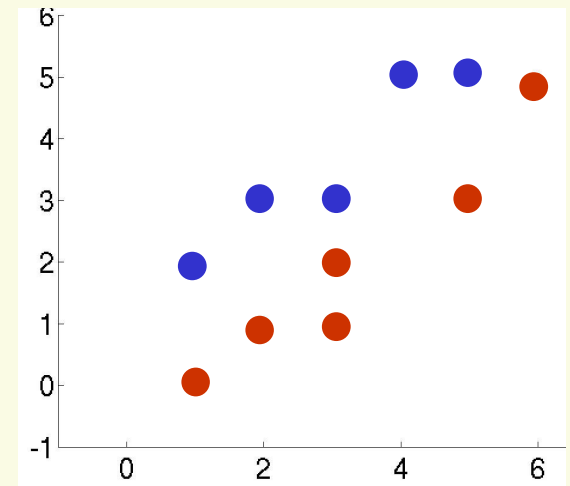
$J(\mathbf{w})$ is maximized for $\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$



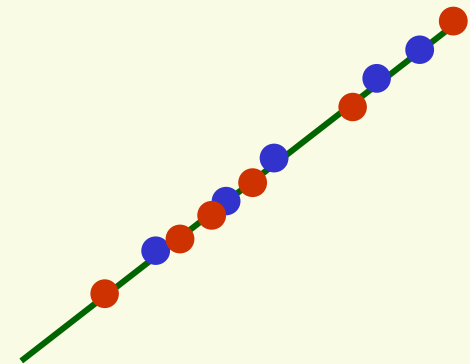
Fisher Linear Discriminant Example

- Data
 - Class 1 has 5 samples $\mathbf{c}_1 = [(1,2), (2,3), (3,3), (4,5), (5,5)]$
 - Class 2 has 6 samples $\mathbf{c}_2 = [(1,0), (2,1), (3,1), (3,2), (5,3), (6,5)]$
- Arrange data in 2 separate matrices

$$\mathbf{c}_1 = \begin{bmatrix} 1 & 2 \\ \vdots & \vdots \\ 5 & 5 \end{bmatrix} \quad \mathbf{c}_2 = \begin{bmatrix} 1 & 0 \\ \vdots & \vdots \\ 6 & 5 \end{bmatrix}$$



- Notice that PCA performs very poorly on this data because the direction of largest variance is not helpful for classification



Fisher Linear Discriminant Example

- First compute the mean for each class

$$\mu_1 = \text{mean}(\mathbf{c}_1) = [3 \quad 3.6] \quad \mu_2 = \text{mean}(\mathbf{c}_2) = [3.3 \quad 2]$$

- Compute scatter matrices $\mathbf{S}_1$ and $\mathbf{S}_2$ for each class

$$\mathbf{S}_1 = 4 * \text{cov}(\mathbf{c}_1) = \begin{bmatrix} 10 & 8.0 \\ 8.0 & 7.2 \end{bmatrix} \quad \mathbf{S}_2 = 5 * \text{cov}(\mathbf{c}_2) = \begin{bmatrix} 17.3 & 16 \\ 16 & 16 \end{bmatrix}$$

- Within the class scatter:

$$\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2 = \begin{bmatrix} 27.3 & 24 \\ 24 & 23.2 \end{bmatrix}$$

- it has full rank, don't have to solve for eigenvalues

- The inverse of $\mathbf{S}_W$ is $\mathbf{S}_W^{-1} = \text{inv}(\mathbf{S}_W) = \begin{bmatrix} 0.39 & -0.41 \\ -0.41 & 0.47 \end{bmatrix}$

- Finally, the optimal line direction $\mathbf{v}$

$$\mathbf{v} = \mathbf{S}_W^{-1}(\mu_1 - \mu_2) = \begin{bmatrix} -0.79 \\ 0.89 \end{bmatrix}$$

Connection to MVN Maximum Likelihood

$J(\mathbf{w})$ is maximized for $\mathbf{w} \propto \mathbf{S}_w^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$

- Recall that if the two distributions are normal with the same covariance Σ , the maximum likelihood classifier is linear, with

$$\mathbf{w} \propto \Sigma^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$$

- Further, note that S_w is proportional to the maximum likelihood estimator for Σ .
- Thus FLD is equivalent to assuming MVN distributions with common covariance.

Connection to Least-Squares

45

Probability & Bayesian Inference

Change coding scheme used in least-squares method to

$$t_n = \frac{N}{N_1} \text{ for } C_1$$

$$t_n = -\frac{N}{N_2} \text{ for } C_2$$

Then one can show that the ML $\mathbf{w}$ satisfies

$$\mathbf{w} \propto \mathbf{S}_w^{-1} (\mathbf{m}_2 - \mathbf{m}_1)$$

Outline

49

Probability & Bayesian Inference

- The Perceptron Algorithm
- Least-Squares Classifiers
- Fisher's Linear Discriminant
- **Logistic Classifiers**

❖ LOGISTIC DISCRIMINATION

- Let an M -class task, $\omega_1, \omega_2, \dots, \omega_M$. In logistic discrimination, the logarithm of the **likelihood ratios** are modeled via **linear** functions, i.e.,

$$\ln \left(\frac{P(\omega_i | \underline{x})}{P(\omega_M | \underline{x})} \right) = w_{i,0} + \underline{w}_i^T \underline{x}, \quad i = 1, 2, \dots, M-1$$

- Taking into account that

$$\sum_{i=1}^M P(\omega_i | \underline{x}) = 1$$

it can be easily shown that the above is equivalent with modeling posterior probabilities as:

$$P(\omega_M | \underline{x}) = \frac{1}{1 + \sum_{i=1}^{M-1} \exp(w_{i,0} + \underline{w}_i^T \underline{x})}$$

$$P(\omega_i | \underline{x}) = \frac{\exp(w_{i,0} + \underline{w}_i^T \underline{x})}{1 + \sum_{i=1}^{M-1} \exp(w_{i,0} + \underline{w}_i^T \underline{x})}, i = 1, 2, \dots, M-1$$

➤ For the two-class case it turns out that

$$P(\omega_2 | \underline{x}) = \frac{1}{1 + \exp(w_0 + \underline{w}^T \underline{x})}$$

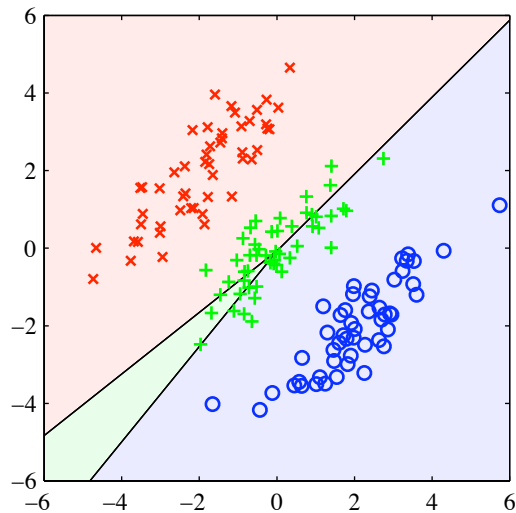
$$P(\omega_1 | \underline{x}) = \frac{\exp(w_0 + \underline{w}^T \underline{x})}{1 + \exp(w_0 + \underline{w}^T \underline{x})}$$

- The unknown parameters $\underline{w}_i, w_{i,0}, i = 1, 2, \dots, M-1$ are usually estimated by **maximum likelihood** arguments.
- Logistic discrimination is a useful tool, since it allows linear modeling and at the same time **ensures** posterior probabilities **to add to one**.

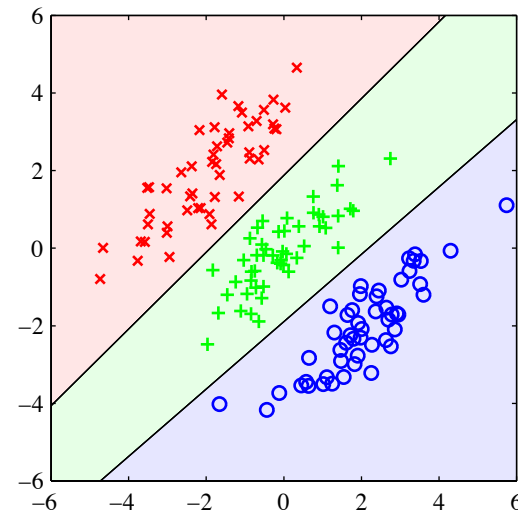
Example

58

Probability & Bayesian Inference



Least-Squares



Logistic